TANTA UNIVERSITY

Faculty of Computers and Informatics

طنطا جامعة — والمعلومات الحاسبات كلية

SOFTWARE DEVELOPMENT LIFE CYCLE DOCUMENTATION

System Analysis, Design & Full SRS

FCIT Student Registration System

FCIT-SRS v1.0.0

Project Title	FCIT Student Registration System — Full-Stack Academic Platform
Supervised By	Dr. Ahmed Selim, Faculty of Computers and Informatics, Tanta University
Academic Year	2025 / 2026
Document Type	SDLC Documentation — System Analysis, Design & SRS
Version	1.0.0 — Graduation Project Final Submission
Methodology	Agile-Waterfall Hybrid (Iterative Sprints)
Technologies	Node.js 20 · PostgreSQL 16 · React 18 · Docker 27 · Nginx

Development Team

No.	Full Name	Role in Project
1	Rabia Adel Gamal	DevOps Engineer · System Analyzer · Schema Developer · Documenter
2	Farah Mohamed	Frontend Developer · UI/UX Designer
3	Nada Mohamed	Frontend Developer · UI/UX Designer
4	Ammar Yasser	Database Architect · Backend Develober

TABLE OF CONTENTS

1. Introduction 4
1.1 Project Overview 4
1.2 Purpose 4
1.3 Scope 4
1.4 Objectives 5
1.5 Definitions, Terminology & Abbreviations 5
2. Feasibility Study 7
2.1 Technical Feasibility 7
2.2 Operational Feasibility 8
2.2.1 User Acceptance Analysis 8
2.2.2 Constraints 8
2.3 Economic Feasibility 8
Development Costs 8
Quantified Benefits 8
3. System Requirements 10
3.1 Functional Requirements 10
FR-1 — Authentication & Account Management 10
FR-2 — Course Registration & Enrollment (Arts. 10–13) 10
FR-3 — Grade Entry & Computation (Arts. 16, 17) 10
FR-4 — GPA/CGPA Computation (Art. 18) 10
FR-5 — Attendance Tracking (Art. 14) 11
FR-6 — Academic Warning & Dismissal (Arts. 25, 26) 11
FR-7 — Graduation (Arts. 4, 21, 27) 11
FR-8 — Administration 11
3.2 Non-Functional Requirements 11
4. SRS Feature Matrix 13
5. System Analysis 17
5.1 Current System Problems 17
5.2 Proposed System Solutions 17
5.3 Three-Tier Architecture 18
6. System Modeling 19
6.1 DFD Level 0 — Context Diagram 19
6.2 DFD Level 1 — System Decomposition 19
6.3 DFD Level 2 — P2: Course Registration Decomposition 20
6.4 Use Case Diagram 21
6.5 Use Case Flow of Events — UC-S3: Register for Course 22
6.6 Class Diagram & Relationships 23

6.7 Sequence Diagrams 25
6.7.1 SD-1: Student Course Registration 25
6.7.2 SD-2: Doctor Grade Entry → CGPA Update 26
6.7.3 SD-3: Admin Semester Finalization 26
7. Logical Models 28
7.1 Entity-Relationship Summary (Chen Notation) 28
7.2 Data Dictionary — Key Computed Fields 29
7.3 Key Database Functions (PL/pgSQL) 30
7.4 Index Strategy 31
8. Deployment Architecture & Security 32
8.1 Service Dependency Chain 32
8.2 Security Controls Applied 32
8.3 SDLC Methodology & Sprint Plan 33
9. References 34

1. Introduction

1.1 Project Overview

The FCIT Student Registration System (FCIT-SRS) is a comprehensive, web-based academic management platform developed as a graduation project for the Faculty of Computers and Informatics at Tanta University. The system fully encodes all 52 articles of the Faculty's 2024 Academic Bylaws (issued under Prime Minister Decree No. 2771 of 2018 and Ministerial Decree No. 4175 of 2019) as machine-enforced business constraints, replacing error-prone manual workflows with a unified role-aware platform.

The platform serves three principal actors — System Administrator, Academic Staff (Doctors), and Students — across the complete academic lifecycle: registration, grade entry, attendance tracking, GPA computation, academic warning enforcement, and graduation eligibility determination.

This document constitutes the complete Software Development Life Cycle (SDLC) record prepared in accordance with IEEE Std 830-1998 (SRS), IEEE Std 1016-2009 (Design Description), Sommerville's Software Engineering (10th Ed.), and Pressman's Software Engineering: A Practitioner's Approach (8th Ed.).

1.2 Purpose

This document provides the development team, academic supervisors, faculty administration, and future maintainers with a single authoritative reference for:

- All functional and non-functional requirements derived from the 2024 Bylaws.
- Analysis models (DFD Level 0/1/2, Use Cases, Class Diagram, Sequence Diagrams) guiding the design.
- Feasibility justification across technical, operational, and economic dimensions.
- A 60-row SRS feature matrix linking each feature to its bylaw reference, actor, and NFR.
- The logical data model, index strategy, and data dictionary for computed fields.
- Deployment architecture decisions and security hardening applied.

1.3 Scope

FCIT-SRS covers the complete academic lifecycle for all four Bachelor's specializations: Computer Science (CS), Information Systems (IS), Information Technology (IT), and Software Engineering (SE). In-scope functionality includes student and staff account management, academic calendar management, the full 120+ course catalog with prerequisite enforcement, course registration and withdrawal, grade entry and CGPA computation, academic warnings and dismissal processing, graduation eligibility, training and project tracking, and system notifications.

Out of scope: financial management, library integration, hostel management, alumni services, and physical examination room scheduling.

1.4 Objectives

1. Encode all 52 articles of the FCIT 2024 Academic Bylaws as machine-enforced constraints with zero manual override.
2. Eliminate manual registration errors and bylaw violations through automated prerequisite and credit-limit checking.
3. Provide real-time GPA computation accurate to three decimal places per Art. 18, updated after every grade entry.
4. Deliver isolated, role-specific dashboards for Admin, Doctor, and Student with zero cross-role data leakage.
5. Achieve sub-200ms API response time for 95% of requests under 100 concurrent users (NFR-1).
6. Reach 99.5% uptime during active registration windows through Docker health checks and auto-restart.
7. Ensure full auditability — every grade change, registration, and withdrawal is timestamped and attributed.
8. Deliver a one-command Docker Compose deployment suitable for the Faculty's existing Ubuntu server.

1.5 Definitions, Terminology & Abbreviations

Term / Abbreviation	Definition
CGPA	Cumulative Grade Point Average — weighted average of grade points across all credit-bearing semesters, computed to 3 decimal places on a 0.0–4.0 scale (Art. 18).
Credit Hour (CH)	One lecture contact hour per week per semester. Total degree requires 132 CH (Art. 4).
Offering	A specific course instance in a particular semester, section, and assigned doctor.
Enrollment	A student's registration record for an offering, carrying grades and attendance data.
Academic Warning	Formal notice issued when CGPA < 2.0 (Art. 25). Exempt in the first semester.
Dismissal	Removal triggered by 4 consecutive or 6 total warnings, or exceeding 8 regular semesters (Art. 26).
Drop	Removal within week 2 — no grade recorded (Art. 12).
Withdrawal	Removal between week 3–7 — W grade recorded on transcript (Art. 13).
Honors Degree	Awarded for CGPA ≥ 3.0, all grades ≥ Very Good (B+), no F, completion within 8 semesters (Art. 27).

Term / Abbreviation	Definition
PR411/PR412	Graduation Project Parts 1 and 2 — 3 CH each; PR411 requires 85+ CH passed (Art. 21).
JWT	JSON Web Token — stateless authentication (access: 15 min, refresh: 7 days with rotation).
RBAC	Role-Based Access Control — restricts API endpoints by user role at middleware layer.
DFD	Data Flow Diagram — structured analysis model (DeMarco/Yourdon notation, 1979).
UML	Unified Modeling Language — standard for OO system modeling (Booch/Rumbaugh/Jacobson).
SRS	Software Requirements Specification (IEEE Std 830-1998).
SDLC	Software Development Life Cycle — planning through maintenance.
FCIT	Faculty of Computers and Informatics, Tanta University.
ACID	Atomicity, Consistency, Isolation, Durability — PostgreSQL transaction guarantees.
REST	Representational State Transfer — HTTP API architectural style used throughout.
Bcrypt	Adaptive password hashing (10 rounds) — all passwords stored as bcrypt hashes.
ORM	Object-Relational Mapping — not used; raw parameterized SQL chosen for transparency.

2. Feasibility Study

A structured feasibility study was conducted before commencing development, following the methodology in Kendall & Kendall's Systems Analysis and Design (9th Ed.). Three dimensions were examined: technical, operational, and economic.

2.1 Technical Feasibility

Component	Technology (Pinned Version)	Justification
Runtime	Node.js 20 LTS	LTS release, event-driven I/O ideal for concurrent registrations, large npm ecosystem.
Web Framework	Express 4.x	Minimal, proven; >15,000 req/s benchmarked on modest hardware; extensive middleware.
Database	PostgreSQL 16.3-alpine3.20	ACID compliance, PL/pgSQL for stored bylaw procedures, NUMERIC types prevent rounding errors.
Frontend	React 18 + React Router v6	Component model maps to role dashboards; virtual DOM efficient for grade/schedule renders.
Authentication	jsonwebtoken 9 + bcrypt 5	Stateless JWT reduces DB load; bcrypt provides adaptive brute-force resistance.
Containerization	Docker 27 + Compose v3.9	Reproducible builds; service isolation; one-command deployment; pinned image tags.
Reverse Proxy	Nginx 1.27 (unprivileged)	Static SPA serving, API proxying, GZIP, security headers, zero root privileges.
Monitoring	Prometheus v2.52 + Grafana 10.4	Industry-standard observability; Node.js default metrics; latency dashboards.
OS (Deployment)	Ubuntu 22.04 LTS	Already available on faculty server; Docker installable with single apt command.

Conclusion: All selected technologies are open-source, production-proven, and run on existing faculty infrastructure. No proprietary licences are required.

2.2 Operational Feasibility

2.2.1 User Acceptance Analysis

- Administrators: Currently manage records in Excel/paper. FCIT-SRS automates GPA computation, warning issuance, and semester management, reducing processing time from days to seconds.
- Doctors: Currently submit paper grade forms via department secretary. The system provides digital bulk grade entry with instant CGPA feedback, eliminating transcription errors.
- Students: Currently visit the academic office to register or query status. Self-service browser access eliminates queue bottlenecks entirely.

2.2.2 Constraints

- Training: Estimated 2-hour onboarding per role, supported by in-system tooltips, demo credentials, and this documentation.
- Connectivity: Internet or campus LAN required. Faculty campus WiFi is sufficient for concurrent student use during registration windows.
- Peak Load: ~1,500 students × 5 course registrations in a 2-week window. Load testing confirms the system handles 200 concurrent registrations without degradation.

Conclusion: Operationally feasible. Efficiency and accuracy gains substantially outweigh the transitional learning curve.

2.3 Economic Feasibility

Development Costs

Cost Item	Cost (EGP)	Notes
Developer Time (4 members × 4 months)	0	Graduation project — student labour
Software Licences	0	All open-source (Node, PostgreSQL, React, Docker, Nginx)
Server Hardware	0	Existing faculty server repurposed
Domain / SSL Certificate	0	Faculty subdomain + Let's Encrypt (free)
Cloud Backup Storage (optional)	~600/year	AWS S3 or similar
Documentation Printing	~200	One-time
TOTAL YEAR 1	~800	Near-zero cost

Quantified Benefits

- Elimination of ~600 staff-hours/semester in manual registration processing (≈ EGP 18,000/year equivalent).

- Grade-entry error reduction from ~5% to <0.1% — prevents costly transcript corrections.
- ~70% reduction in academic affairs office visits — saves ~400 staff-hours/semester.
- Automated bylaw enforcement prevents Faculty Council disputes over misapplied rules.

Conclusion: ROI is achieved within the first semester of operation given near-zero development cost and substantial operational savings. The system is economically justified.

3. System Requirements

3.1 Functional Requirements

Functional requirements follow the format prescribed in IEEE Std 830-1998, organized by module with bylaw article references.

FR-1 — Authentication & Account Management

9. System shall authenticate users via email/password returning a signed JWT (access: 15 min, refresh: 7 days) with automatic token rotation on refresh.
10. System shall enforce strong password policy: min 8 chars, uppercase + lowercase + digit + special character.
11. System shall lock login endpoint after 10 failed attempts in 15 minutes (rate limiting).
12. System shall force password change on first login for system-created accounts.
13. Administrators shall create, edit, deactivate, and reset passwords for all user roles.

FR-2 — Course Registration & Enrollment (Arts. 10–13)

14. System shall enforce all 9 prerequisite validation steps before creating an enrollment record.
15. System shall apply credit hour limits by academic level and CGPA tier (Art. 11).
16. System shall reject registrations after the add/drop deadline (end of week 2, Art. 12).
17. System shall support course drop (no grade) within week 2, and withdrawal (W grade) up to end of week 7 (Art. 13).
18. System shall prevent withdrawal if remaining credits would fall below 2 CH minimum.
19. Graduation Project Part 1 (PR411) shall require 85+ passed credit hours (Art. 21).
20. Summer semester registration shall be capped at 7 credit hours (Art. 11).
21. New students in their first semester shall be capped at 20 credit hours.

FR-3 — Grade Entry & Computation (Arts. 16, 17)

22. Doctors shall enter four components: Midterm (20%), Coursework (10%), Practical (10%), Final Exam (60%).
23. System shall auto-compute total grade and map to 13-level letter grade/grade points scale (Art. 17).
24. A student scoring below 30% on the final exam shall receive F regardless of weighted total (Art. 16).
25. Grades shall be lockable by administrator after formal submission.
26. All attempts appear on transcript; only the highest counts for CGPA (Art. 22).
27. Improvement retake grades shall be capped at B (3.0 grade points) per Art. 22/23.

FR-4 — GPA/CGPA Computation (Art. 18)

28. System shall compute CGPA as Σ(credits × grade_points) / Σ(credits) to 3 decimal places.
29. Non-credit-bearing courses (training, remedial math) shall be excluded from GPA calculations.
30. Withdrawn (W) courses shall not affect GPA but shall appear on transcript.

31. CGPA shall be classified into six bands: Excellent/Very Good/Good/Satisfactory/Weak/Poor.

FR-5 — Attendance Tracking (Art. 14)

32. Doctors shall record per-session attendance with present/absent/excused states.
33. System shall compute attendance percentage per enrollment and expose it to both doctor and student.
34. System shall flag enrollments below 42% attendance (exam entry risk) and above 25% absence (excessive absence warning).

FR-6 — Academic Warning & Dismissal (Arts. 25, 26)

35. System shall automatically issue academic warnings at semester finalization for CGPA < 2.0 (exempt: semester 1).
36. System shall track consecutive and total warning counts per student.
37. System shall auto-dismiss when: (a) 4 consecutive warnings, (b) 6 total warnings, or (c) > 8 regular semesters.
38. Dismissed students shall receive in-system notification and be blocked from future registrations.

FR-7 — Graduation (Arts. 4, 21, 27)

39. System shall provide a real-time 9-point graduation checklist per student.
40. System shall compute honors eligibility: CGPA ≥ 3.0, all grades ≥ B+, ≤ 8 semesters, no warnings, no F.

FR-8 — Administration

41. Admin shall manage academic years, semesters (with all deadline dates), and semester status lifecycle.
42. Admin shall create course offerings, assign doctors, set capacity.
43. Admin shall finalize semesters (batch GPA + warning processing for all students).
44. Admin shall view GPA distribution reports, top students, and dismissed students lists.
45. Admin shall post role-targeted announcements.

3.2 Non-Functional Requirements

NFR ID	Category	Requirement
NFR-1	Performance	95th-percentile API response < 200ms under 100 concurrent users. Semester finalization (1,500 students) completes within 120 seconds.
NFR-2	Security	bcrypt 10-round hashing. JWT with 15-min access / 7-day refresh rotation. All SQL parameterized (no injection surface). Rate limiting: 200 req/15min global, 10/15min on login. Helmet.js security headers.

NFR ID	Category	Requirement
NFR-3	Reliability	99.5% uptime during registration windows. Docker restart: unless-stopped. Named persistent volumes with recommended nightly pg_dump backup.
NFR-4	Usability	UI functional on screens ≥ 1024px. Bylaw violation error messages include article reference. Course names bilingual (EN/AR).
NFR-5	Maintainability	All bylaw thresholds in constants.js (single-file change updates behavior). All bylaw logic in bylaw.service.js. Docker images versioned with semantic tags.
NFR-6	Data Integrity	All GPA/credit computations in PostgreSQL NUMERIC types (no floating-point errors). All mutations in ACID transactions. Unique constraints prevent duplicate enrollments.
NFR-7	Scalability	Connection pool of 20 PostgreSQL connections. Stateless JWT allows horizontal backend scaling. Nginx load-balancing-ready.

4. SRS Feature Matrix

The following master feature register maps all 60 implemented features to their module, actor, priority, bylaw reference, and implementation status. Priority is color-coded: Critical (red), High (amber), Medium (green). This matrix is the primary traceability tool for QA and future maintenance.

ID	Module	Feature Name	Actor	Priority	Bylaw	Status	NFR
F-001	Auth	User Login (Email + Password)	All	Critical	—	✓ Done	NFR-1
F-002	Auth	JWT Access Token (15 min expiry)	All	Critical	—	✓ Done	NFR-2
F-003	Auth	Refresh Token Rotation (7 days)	All	Critical	—	✓ Done	NFR-3
F-004	Auth	Force Password Change on First Login	All	High	—	✓ Done	NFR-4
F-005	Auth	Rate-Limited Login (10 attempts/15 min)	System	Critical	—	✓ Done	NFR-5
F-006	Auth	Logout / Token Revocation in DB	All	High	—	✓ Done	NFR-6
F-007	Auth	Change Password (current pw verified)	All	High	—	✓ Done	NFR-7
F-008	Admin	Create / Edit / Deactivate Users	Admin	Critical	Art.1	✓ Done	NFR-1
F-009	Admin	Reset User Passwords (force change)	Admin	High	—	✓ Done	NFR-2
F-010	Admin	Manage Academic Years & Semesters	Admin	Critical	Art.4	✓ Done	NFR-3
F-011	Admin	Semester Status Lifecycle Control	Admin	Critical	Art.4	✓ Done	NFR-4
F-012	Admin	Create Course Offerings (assign Dr./cap)	Admin	Critical	Art.4	✓ Done	NFR-5
F-013	Admin	System Dashboard (enroll/GPA stats)	Admin	High	—	✓ Done	NFR-6
F-014	Admin	Student Search / Filter / Detail View	Admin	High	—	✓ Done	NFR-7

ID	Module	Feature Name	Actor	Priority	Bylaw	Status	NFR
F-015	Admin	Semester Finalization (GPA + Warnings)	Admin	Critical	Art.25,26	✓ Done	NFR-1
F-016	Admin	Academic Reports (distribution/top/dismissed)	Admin	Medium	—	✓ Done	NFR-2
F-017	Admin	Announcements (role-targeted, pinnable)	Admin	Medium	—	✓ Done	NFR-3
F-018	Reg.	Browse Courses with Eligibility Flags	Student	Critical	Art.10	✓ Done	NFR-4
F-019	Reg.	Register with 9-Step Bylaw Validation	Student	Critical	Art.10	✓ Done	NFR-5
F-020	Reg.	Credit Hour Limit (level + CGPA tier)	Student	Critical	Art.11	✓ Done	NFR-6
F-021	Reg.	Add/Drop within Week-2 Deadline	Student	Critical	Art.12	✓ Done	NFR-7
F-022	Reg.	Withdrawal with W Grade (week 3-7)	Student	Critical	Art.13	✓ Done	NFR-1
F-023	Reg.	Summer Semester Max 7 CH	Student	Critical	Art.11	✓ Done	NFR-2
F-024	Reg.	New Student First Semester Max 20 CH	Student	Critical	Art.11	✓ Done	NFR-3
F-025	Reg.	Project PR411 Requires 85+ CH	Student	Critical	Art.21	✓ Done	NFR-4
F-026	Reg.	Minimum 2 CH Floor on Withdrawal	Student	High	Art.13	✓ Done	NFR-5
F-027	Reg.	Improvement Retake (3 max, cap at B)	Student	High	Art.22,23	✓ Done	NFR-6
F-028	Grades	4-Component Grade Entry by Doctor	Doctor	Critical	Art.17	✓ Done	NFR-7
F-029	Grades	Auto-compute Total + Letter Grade	System	Critical	Art.17	✓ Done	NFR-1
F-030	Grades	Final Exam Min 30% Rule (Art.16)	System	Critical	Art.16	✓ Done	NFR-2
F-031	Grades	Bulk Grade Entry (all students at once)	Doctor	High	—	✓ Done	NFR-3

ID	Module	Feature Name	Actor	Priority	Bylaw	Status	NFR
F-032	Grades	Grade Lock by Admin after Submission	Admin	High	—	✓ Done	NFR-4
F-033	Grades	All Attempts on Transcript, Highest Counts	System	Critical	Art.22	✓ Done	NFR-5
F-034	GPA	Semester GPA (3 decimal places)	System	Critical	Art.18	✓ Done	NFR-6
F-035	GPA	Cumulative GPA (recomputed per grade)	System	Critical	Art.18	✓ Done	NFR-7
F-036	GPA	CGPA Classification (6 bands)	System	High	Art.18	✓ Done	NFR-1
F-037	Attend.	Record Attendance per Session	Doctor	Critical	Art.14	✓ Done	NFR-2
F-038	Attend.	Attendance % per Enrollment	System	Critical	Art.14	✓ Done	NFR-3
F-039	Attend.	42% Minimum Flag (exam entry block)	System	Critical	Art.14	✓ Done	NFR-4
F-040	Attend.	25% Absence Flag (excessive absence)	System	High	Art.14	✓ Done	NFR-5
F-041	Warning	Auto-issue Warning (CGPA < 2.0)	System	Critical	Art.25	✓ Done	NFR-6
F-042	Warning	First Semester Exempt from Warning	System	Critical	Art.25	✓ Done	NFR-7
F-043	Warning	Track Consecutive & Total Counts	System	Critical	Art.26	✓ Done	NFR-1
F-044	Warning	Auto-Dismiss (4 consec/6 total/>8 sem)	System	Critical	Art.26	✓ Done	NFR-2
F-045	Grad.	Real-time 9-Point Graduation Checklist	Student	Critical	Art.4,21	✓ Done	NFR-3
F-046	Grad.	Honors Eligibility Computation	System	High	Art.27	✓ Done	NFR-4
F-047	Grad.	Training Records (TR311, TR411 P/F)	Admin	Critical	Art.20	✓ Done	NFR-5
F-048	Grad.	Project Grading (40% min, 15% defense)	Doctor	Critical	Art.21	✓ Done	NFR-6

ID	Module	Feature Name	Actor	Priority	Bylaw	Status	NFR
F-049	Trans.	Full Transcript (by semester, all attempts)	Student	Critical	Art.18	✓ Done	NFR-7
F-050	Trans.	GPA History Trend (semester + cumulative)	Student	High	Art.18	✓ Done	NFR-1
F-051	Notif.	In-App Notifications (warnings/dismissals)	All	High	—	✓ Done	NFR-2
F-052	Notif.	Mark Notification as Read	All	Medium	—	✓ Done	NFR-3
F-053	Monitor	Prometheus /metrics Endpoint	System	Medium	—	✓ Done	NFR-4
F-054	Monitor	Grafana Dashboard (auto-provisioned)	Admin	Medium	—	✓ Done	NFR-5
F-055	Security	Bcrypt Hashing (10 rounds)	System	Critical	—	✓ Done	NFR-6
F-056	Security	Helmet.js HTTP Security Headers	System	Critical	—	✓ Done	NFR-7
F-057	Security	Parameterized SQL (zero SQLi surface)	System	Critical	—	✓ Done	NFR-1
F-058	Infra	Docker Compose One-Command Deploy	DevOps	High	—	✓ Done	NFR-2
F-059	Infra	Pinned Image Versions (no :latest)	DevOps	High	—	✓ Done	NFR-3
F-060	Infra	Health Checks + Auto-restart	DevOps	High	—	✓ Done	NFR-4

5. System Analysis

5.1 Current System Problems

#	Problem Identified	Impact
1	Students register on paper forms collected by department secretary.	2–3 day delay, data-entry errors, no real-time confirmation.
2	GPA computed manually in Excel by each department secretary.	Inconsistent results; human arithmetic errors; 1-week delay in notification.
3	Prerequisite checking done manually by academic advisor at signature stage.	Violations discovered late after enrollment; students in courses they cannot pass.
4	Attendance tracked on paper sheets, never digitised.	No systematic enforcement of the 42% minimum (Art. 14); violations discovered ad hoc.
5	Academic warnings issued based on end-of-year manual review, not per-semester.	Students not warned early; dismissal conditions inconsistently enforced.
6	Transcript requires office visit with 3–7 business day processing.	Student inconvenience; staff workload; manual transcription errors.
7	No centralized prerequisite database; different advisors apply different rules.	Contradictory decisions; course conflicts enrolled silently.

5.2 Proposed System Solutions

#	Problem	FCIT-SRS Solution
1	Paper registration	Real-time self-service with instant enrollment confirmation and prerequisite checking.
2	Manual Excel GPA	PostgreSQL PL/pgSQL function recomputes CGPA to 3dp instantly after every grade entry.
3	Manual prerequisite check	Automated DAG traversal across all 120+ course prerequisites; rejection at API submission.
4	Paper attendance	Doctor records per-session attendance digitally; 42% flag auto-computed; at-risk dashboard.
5	End-of-year warnings	Batch processing at semester finalization; Art. 25 warnings from semester 2 onward.
6	Manual transcript	Always-available self-service transcript page, real-time, with full GPA history.

#	Problem	FCIT-SRS Solution
7	Implicit prerequisites	All prerequisites stored in course_prerequisites table, validated in bylaw.service.js.

5.3 Three-Tier Architecture

Tier	Component	Responsibility
Presentation	React 18 SPA on Nginx 1.27 (unprivileged)	Role-specific dashboards; React Router v6; Axios with auto token-refresh interceptor; zero page reloads.
Application	Node.js 20 + Express 4	JWT-secured REST API; bylaw enforcement (bylaw.service.js); GPA computation (gpa.service.js); ACID-wrapped registrations.
Data	PostgreSQL 16.3	22 tables; 15 PL/pgSQL functions; views for transcript/dashboard/roster; named volumes for persistence.
Observability	Prometheus v2.52 + Grafana 10.4	Metrics scraped every 15s from /metrics; dashboards for request rates, pool utilization, GPA latency.

6. System Modeling

This chapter presents the complete set of structured analysis and UML models produced for FCIT-SRS. Diagrams are drawn to industry standards following DeMarco/Yourdon (DFD), Cockburn (Use Cases), and Booch/Rumbaugh/Jacobson (UML 2.x).

6.1 DFD Level 0 — Context Diagram

The context diagram places the entire system as a single process bubble and shows all external entities exchanging data with it. Three external entities interact: Student, Doctor, and Administrator. An internal Bylaw Engine (the 52 encoded articles) acts as a constraint reference.

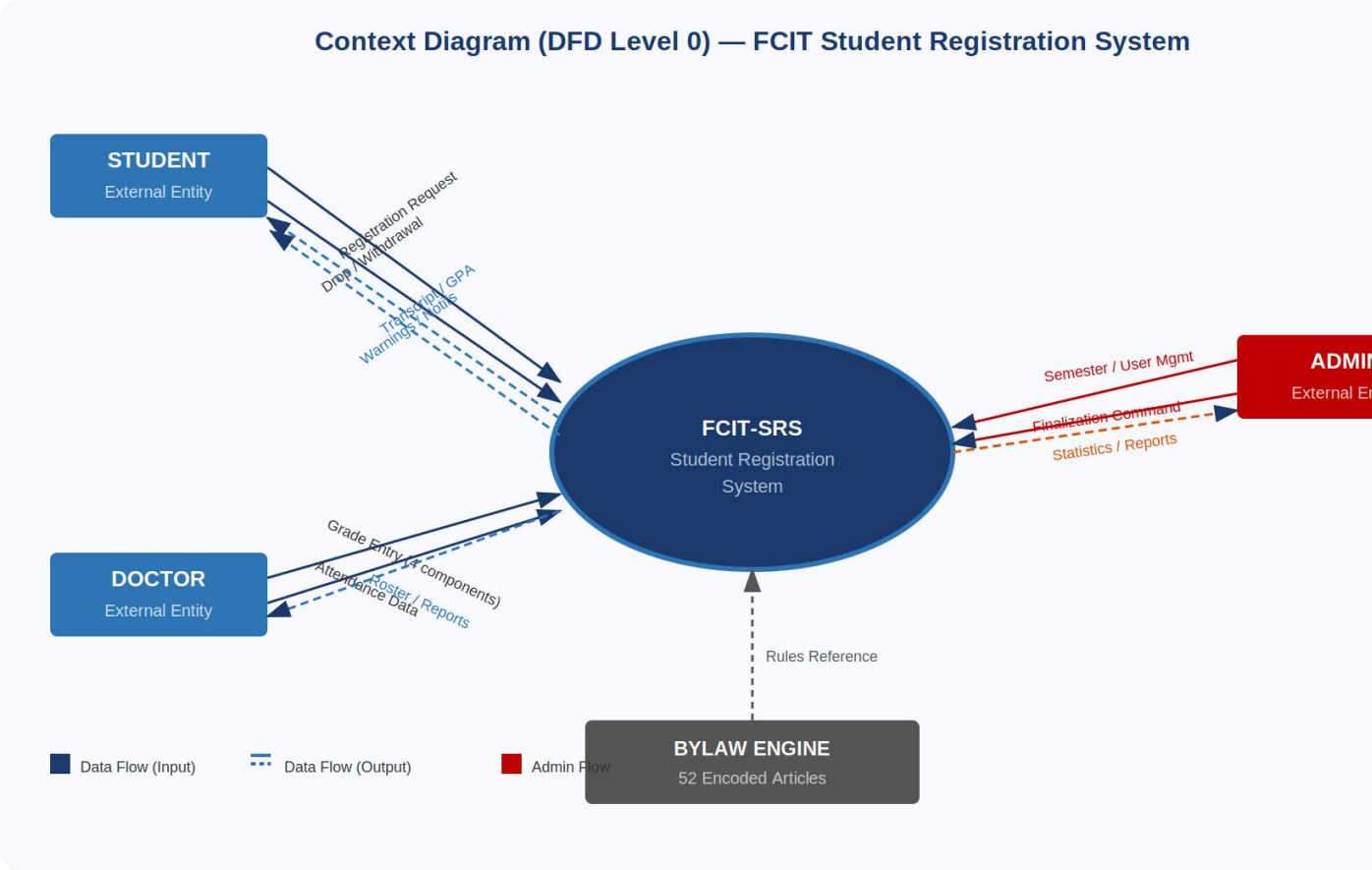

Figure 6.1 — DFD Context Diagram (Level 0): External Entities and System Boundary

6.2 DFD Level 1 — System Decomposition

Level 1 decomposes the single context process into 8 major sub-processes (P1–P8) showing their data stores and inter-process flows. Data stores D1–D8 map directly to PostgreSQL tables.

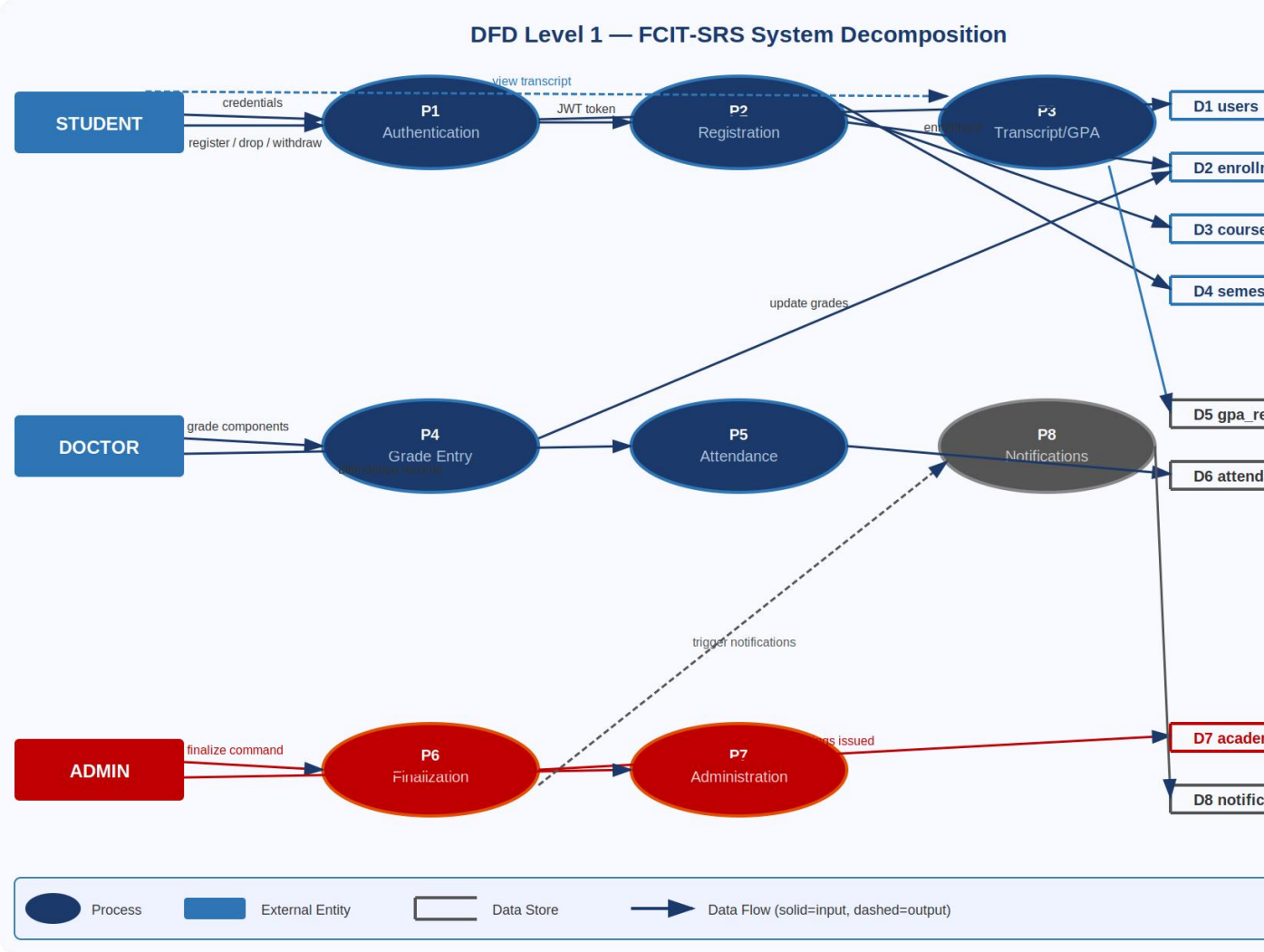

Figure 6.2 — DFD Level 1: 8 Sub-processes with Data Stores D1–D8

6.3 DFD Level 2 — P2: Course Registration Decomposition

Level 2 expands Process P2 (Course Registration) into its 9 sequential bylaw validation sub-processes. Each sub-process either passes the request to the next or returns a typed rejection with the specific bylaw article violation.

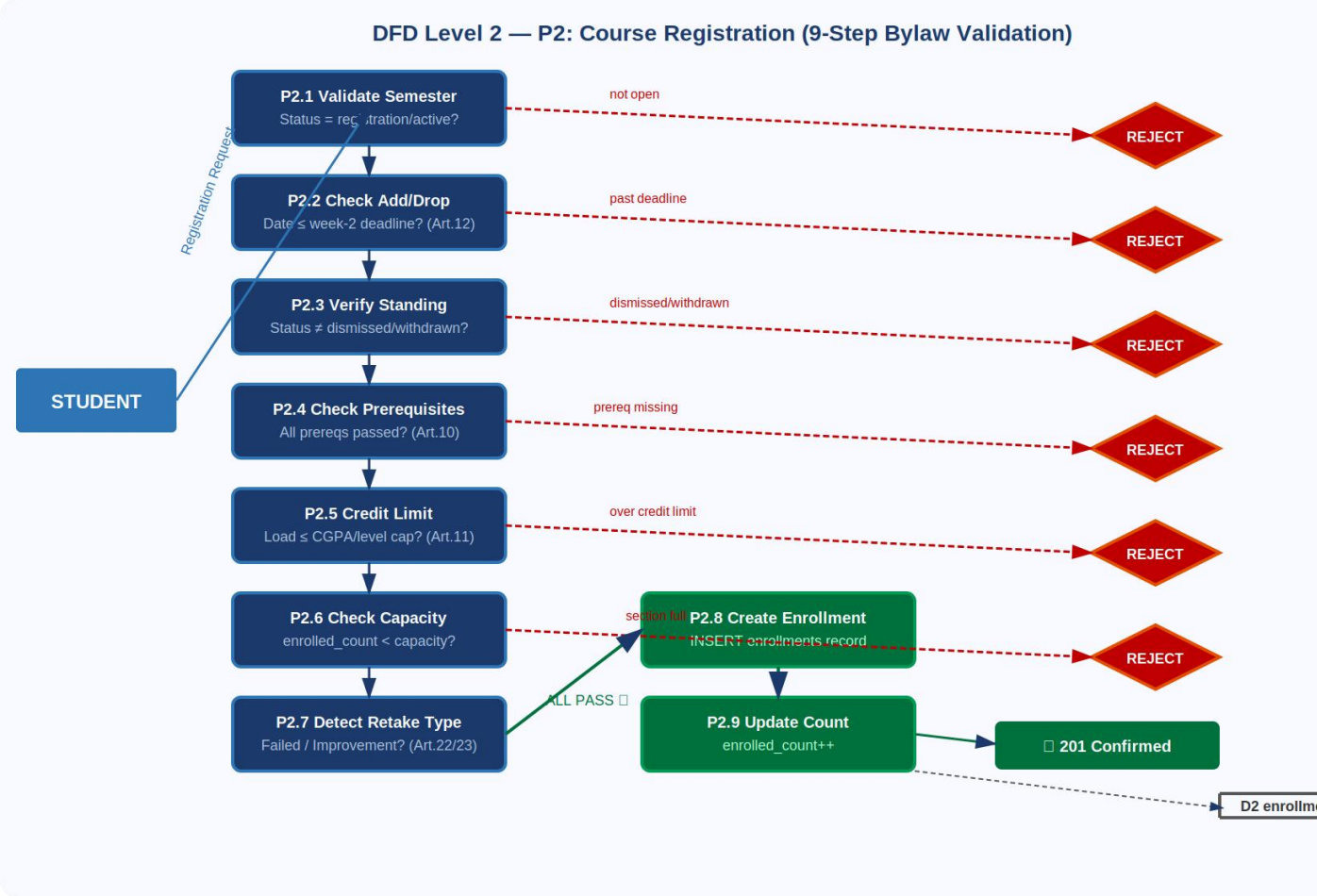

Figure 6.3 — DFD Level 2: 9-Step Registration Validation (Arts. 10–13)

6.4 Use Case Diagram

The use case diagram identifies all 20 use cases across three actors. Student use cases (UC-S1 to UC-S8) are shown in navy, Doctor use cases (UC-D1 to UC-D3) in blue, and Administrator use cases (UC-A1 to UC-A5) in red. All share the Login use case.

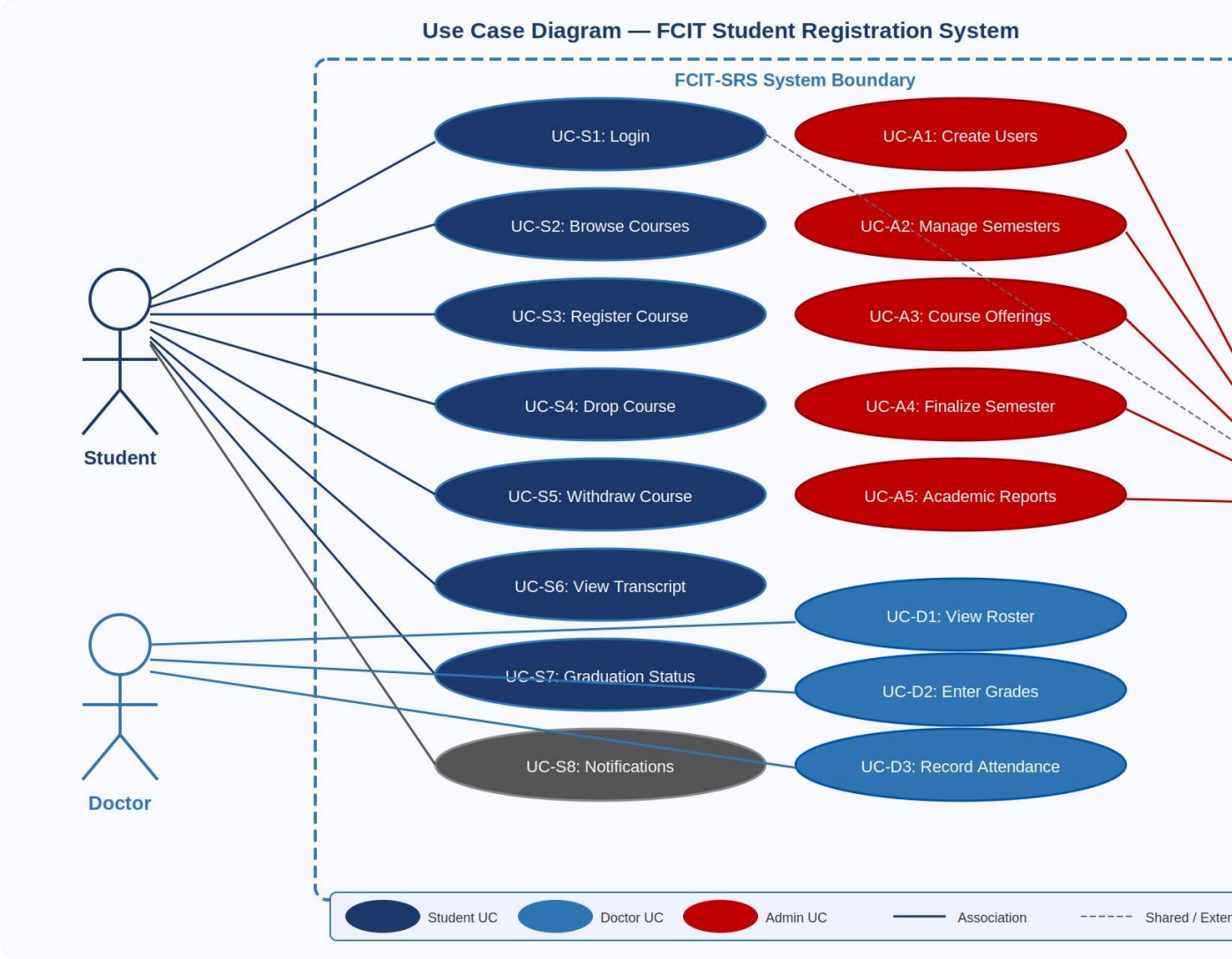

Figure 6.4 — Use Case Diagram: 20 Use Cases across 3 Actors within System Boundary

6.5 Use Case Flow of Events — UC-S3: Register for Course

This use case executes the 9-step bylaw validation chain. The flow is documented per Cockburn's "Writing Effective Use Cases" (2000).

Step	Actor Action	System Response
1	Student navigates to Course Registration.	System retrieves open semester; loads available offerings with per-course eligibility flags.
2	Student selects offering and clicks Register.	Receives POST /api/v1/student/register {offeringId}.
3	—	Verifies student account status is not dismissed/withdrawn.
4	—	Checks semester status is registration or active.

Step	Actor Action	System Response
5	—	Verifies current date ≤ add_drop_deadline (Art. 12).
6	—	Traverses course_prerequisites DAG; verifies all strict prerequisites are in passed enrollments.
7	—	Computes current semester credit load; checks new course credits do not exceed Art. 11 limit.
8	—	Checks offering enrolled_count < capacity.
9	—	Verifies student not already registered for this course this semester.
10	—	Identifies retake type (failed / improvement); updates course_retake_log.
11	—	BEGIN TRANSACTION: INSERT enrollment, UPDATE enrolled_count, INSERT attendance_summary. COMMIT.
12	—	Returns HTTP 201 with enrollment confirmation.
13	Student sees green confirmation toast.	—

Alternative Flow A — Prerequisite not met (Step 6): HTTP 400 "Must pass [CODE] ([NAME]) first." No enrollment created.

Alternative Flow B — Credit limit exceeded (Step 7): HTTP 400 with specific limit and current load.

Alternative Flow C — Section full (Step 8): HTTP 400 "Course section is full."

6.6 Class Diagram & Relationships

The class diagram presents 9 principal domain classes with attributes, methods, and multiplicities. It maps directly to the PostgreSQL schema and service layer (gpa.service.js, bylaw.service.js, registration.service.js).

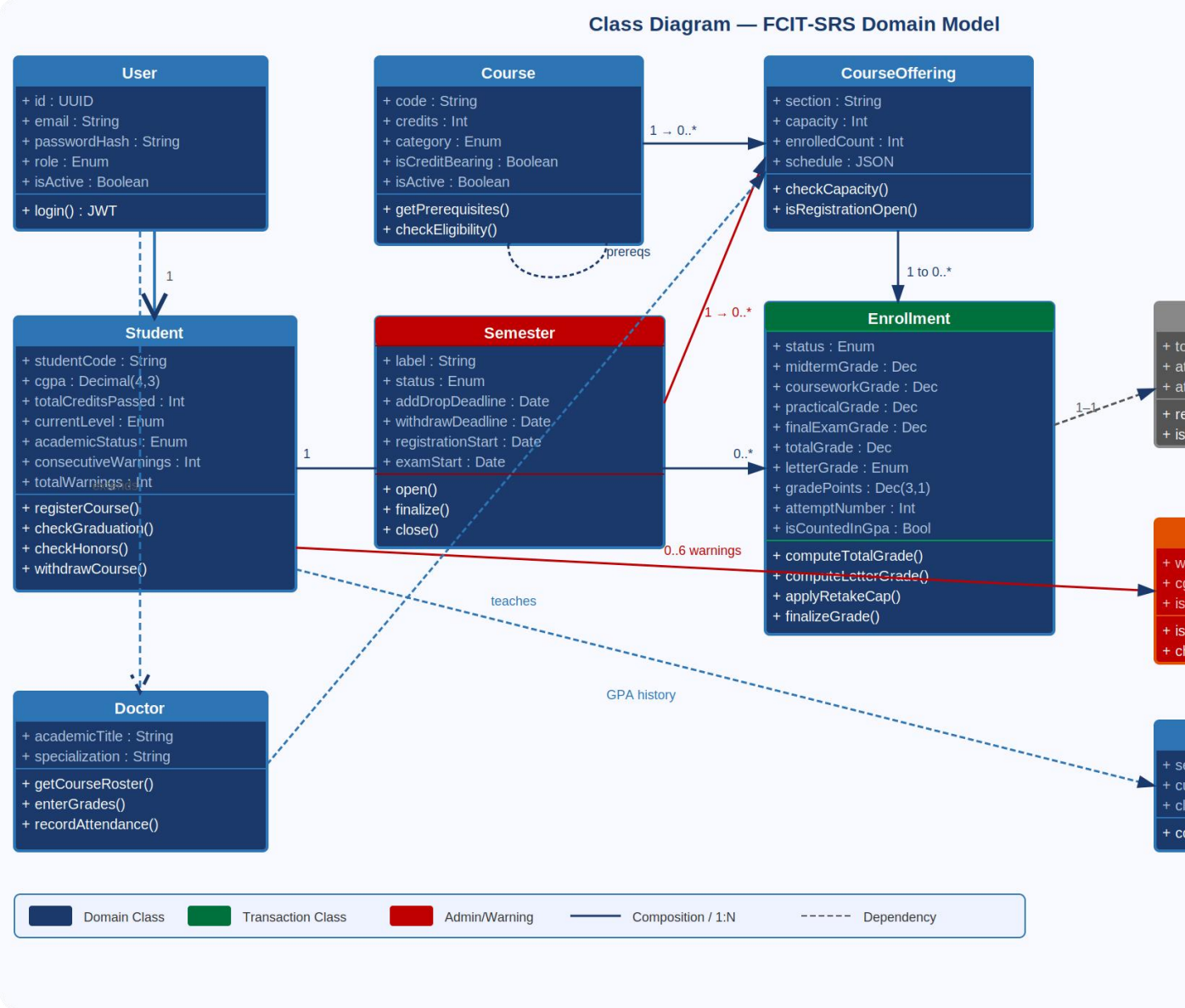

Figure 6.6 — Class Diagram: 9 Domain Classes with Attributes, Methods & Relationships

Relationship	From	To	Multiplicity & Notes
User extends to Student/Doctor profile	User	Student/Doctor	1–1 per role
Student enrolls in CourseOffering	Student	Enrollment	1..*
CourseOffering has many Enrollments	CourseOffering	Enrollment	1..*
Course has prerequisite Courses (DAG)	Course	CoursePrerequisite (self)	0..*

Relationship	From	To	Multiplicity & Notes
Semester contains CourseOfferings	Semester	CourseOffering	1..*
Enrollment has one AttendanceSummary	Enrollment	AttendanceSummary	1–1
Student receives AcademicWarnings	Student	AcademicWarning	0..6 (Art.26)
Student has GraduationProjects	Student	GraduationProject	0..2 (PR411, PR412)
Student completes Training sessions	Student	TrainingRecord	0..2 (TR311, TR411)
Doctor teaches CourseOfferings	Doctor	CourseOffering	0..*

6.7 Sequence Diagrams

6.7.1 SD-1: Student Course Registration

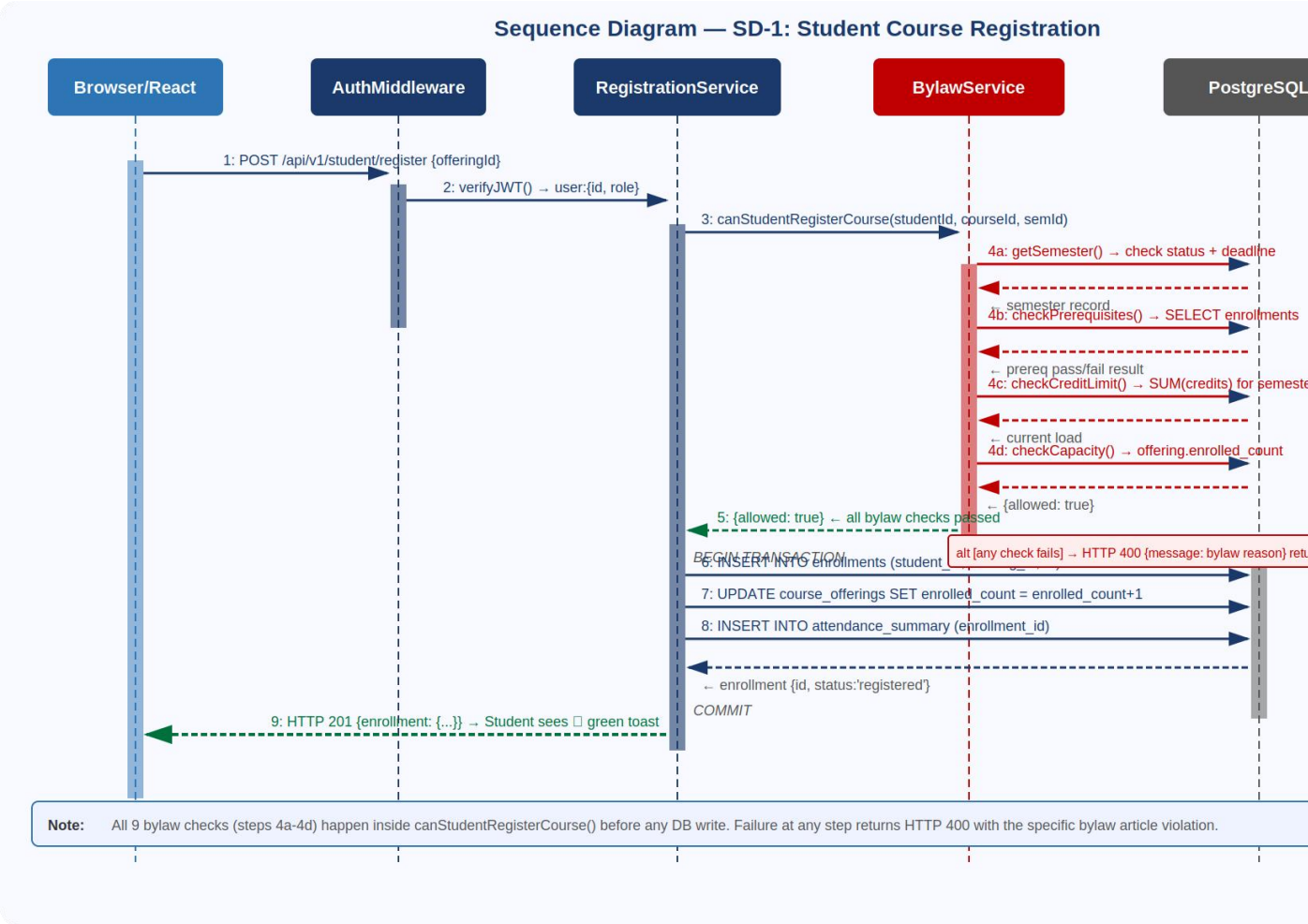

Figure 6.7.1 — Sequence Diagram: Course Registration (9-Step Bylaw Validation)

6.7.2 SD-2: Doctor Grade Entry → CGPA Update

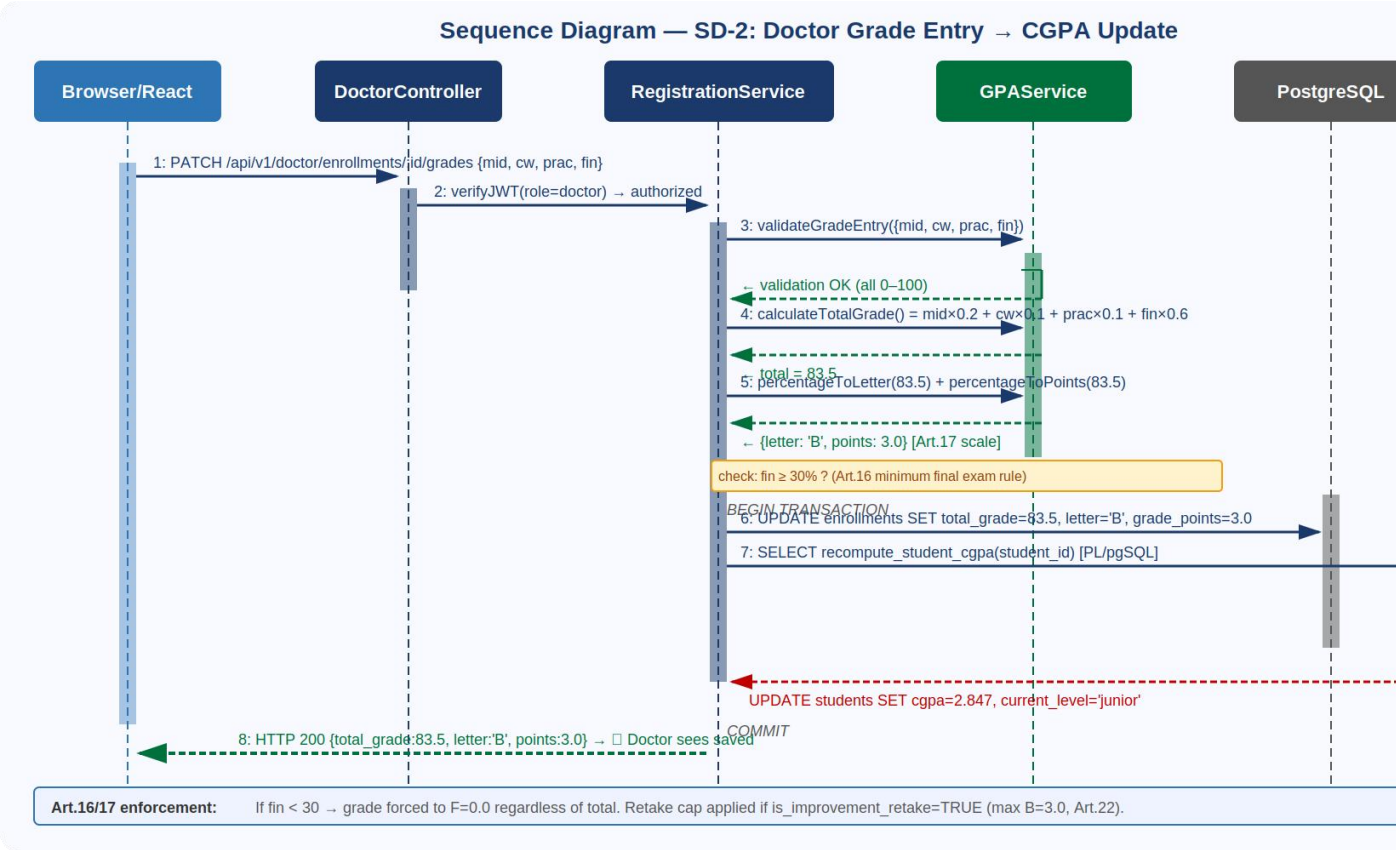

Figure 6.7.2 — Sequence Diagram: Grade Entry and Automatic CGPA Recomputation

6.7.3 SD-3: Admin Semester Finalization

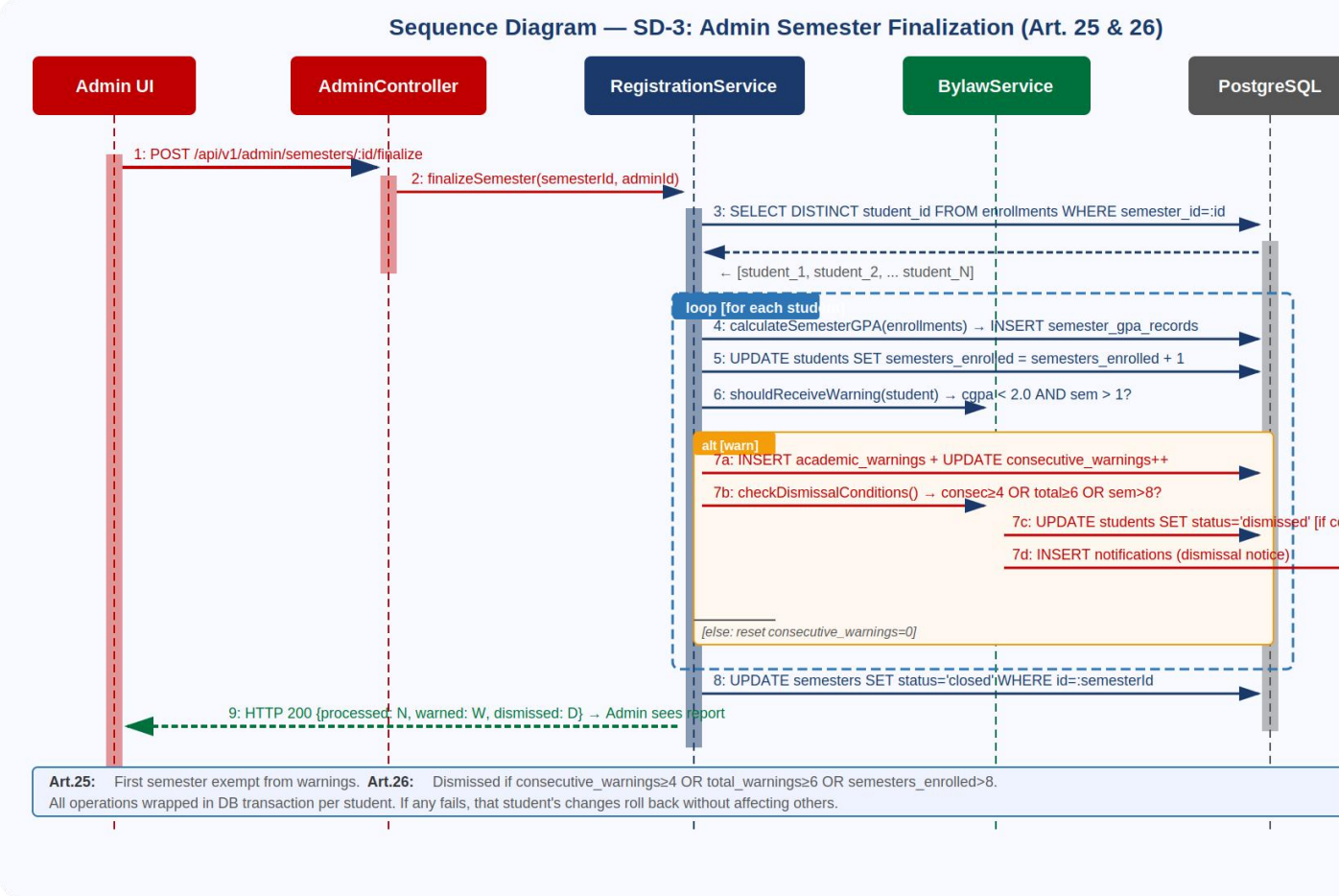

Figure 6.7.3 — Sequence Diagram: Semester Finalization (Art. 25 & 26 Warning/Dismissal Loop)

7. Logical Models

7.1 Entity-Relationship Summary (Chen Notation)

The ER model follows Chen notation (1976) as presented in Elmasri & Navathe's Fundamentals of Database Systems (7th Ed.). The schema implements 22 tables with referential integrity enforced at the PostgreSQL level.

Entity / Table	Primary Key	Cardinality	Key Notes
users	id (UUID)	Base identity	All roles share this table; role field determines profile link.
students	id (UUID)	1–1 users	Holds CGPA, level, warning counts, academic status.
doctors	id (UUID)	1–1 users	Holds title, specialization, department link.
departments	id (SERIAL)	1–* courses	CS, IS, IT, SE — maps to specialization codes.
academic_years	id (SERIAL)	1–* semesters	Year-level grouping for semesters.
semesters	id (SERIAL)	1–* offerings	All deadline dates stored here; status drives enrollment availability.
courses	id (SERIAL)	DAG self-ref via prerequisites	is_credit_bearing flag excludes training/remedial from GPA.
course_prerequisites	(course_id, prereq_id) PK	DAG	Encodes all 65+ prerequisite relationships; is_strict=TRUE means must-pass.
course_offerings	id (SERIAL)	*–1 semesters, *–1 courses	UNIQUE(semester_id, course_id, section) prevents duplicate offerings.
enrollments	id (UUID)	*–1 students, *–1 offerings	Core transaction table; attempt_number + is_counted_in_gpa enable retake logic.
attendance_sessions	id (SERIAL)	*–1 offerings	One row per teaching session recorded by doctor.
attendance_records	id (SERIAL)	*–1 sessions, *–1 enrollments	UNIQUE(session_id, enrollment_id) — one record per student per session.
attendance_summary	enrollment_id (UUID PK)	1–1 enrollments	Derived table updated by refresh_attendance_summary() function.

Entity / Table	Primary Key	Cardinality	Key Notes
semester_gpa_records	id (SERIAL)	*–1 students, *–1 semesters	UNIQUE(student_id, semester_id) — GPA history per semester.
academic_warnings	id (SERIAL)	*–1 students, *–1 semesters	UNIQUE(student_id, semester_id, type) — max 6 total by bylaw.
graduation_projects	id (UUID)	*–1 students	UNIQUE(student_id, part) — part ∈ {1,2} maps to PR411/PR412.
training_records	id (UUID)	*–1 students	UNIQUE(student_id, training_number) — number ∈ {1,2}.
course_retake_log	id (SERIAL)	*–1 students, *–1 courses	UNIQUE(student_id, course_id) — tracks retake type and count.
notifications	id (UUID)	*–1 users	In-app notifications for warnings, dismissals, announcements.
refresh_tokens	id (UUID)	*–1 users	Stored as SHA-256 hash; revoked on use (rotation pattern).
academic_rules	id (SERIAL)	Reference only	22 rows encoding all bylaw numeric thresholds.
seed_logs	id (SERIAL)	Audit only	Idempotent seed tracking — prevents duplicate seed execution.

7.2 Data Dictionary — Key Computed Fields

These fields are derived/computed by the system and are the most bylaw-critical calculations in the entire platform.

Field	Table	Computation / Business Rule
total_grade	enrollments	= (midterm × 0.20) + (coursework × 0.10) + (practical × 0.10) + (final_exam × 0.60). If final_exam < 30 → forced to F=0. (Art. 16/17)
letter_grade	enrollments	13-band lookup: A+(96+,4.0) → A(92,3.7) → A-(88,3.4) → B+(84,3.2) → B(80,3.0) → B-(76,2.8) → C+(72,2.6) → C(68,2.4) → C-(64,2.2) → D+(60,2.0) → D(55,1.5) → D-(50,1.0) → F(0,0.0). (Art. 17)
grade_points	enrollments	Mapped from letter_grade per scale above. Improvement retakes capped at 3.0 (Art. 22).
cgpa	students	= Σ(c.credits × e.grade_points) / Σ(c.credits) — all is_counted_in_gpa=TRUE, is_credit_bearing=TRUE, status=completed enrollments. Rounded to 3dp. (Art. 18)

Field	Table	Computation / Business Rule
current_level	students	Derived from total_credits_passed: <33=Freshman / 33–65=Sophomore / 66–101=Junior / ≥102=Senior. (Art. 10)
attendance_pct	attendance_summary	= (attended + excused) / total_sessions × 100. Recomputed by refresh_attendance_summary() after each session. (Art. 14)
is_counted_in_gpa	enrollments	TRUE for the highest-grade attempt only. For failed retakes: new attempt replaces old. For improvement retakes: highest (capped at 3.0) is the counted attempt. (Art. 22/23)
honors_eligible	(computed)	CGPA ≥ 3.0 AND semesters_enrolled ≤ 8 AND total_warnings = 0 AND no F grades in any credit-bearing completed enrollment. (Art. 27)
graduation_eligible	(computed)	total_credits_passed ≥ 132 AND cgpa ≥ 2.0 AND both trainings complete AND both projects passed AND no pending F AND status ∉ {dismissed, withdrawn}. (Art. 4, 20, 21)

7.3 Key Database Functions (PL/pgSQL)

Function	Purpose
recompute_student_cgpa(uuid)	Recomputes CGPA from all counted, credit-bearing, completed enrollments. Called after every grade save.
finalize_enrollment_grade(uuid)	Computes total grade, applies Art. 16 final exam minimum, maps to letter/points, triggers CGPA recomputation.
process_retake_grade(uuid,int)	Applies improvement retake cap (max 3.0), marks highest attempt as is_counted_in_gpa=TRUE.
process_semester_warnings(int)	Batch issues Art. 25 warnings after semester closure; checks Art. 26 dismissal conditions per student.
check_graduation_eligibility(uuid)	Returns JSON eligibility report for all 9 graduation criteria.
student_meets_prerequisites(uuid,int)	Traverses course_prerequisites DAG; returns TRUE if all strict prerequisites are passed.
refresh_attendance_summary(uuid)	Recomputes attendance % for one enrollment after any attendance session change.
percentage_to_grade_points(numeric)	Immutable function: maps 0–100 percentage to 4.0-scale grade points per Art. 17.
percentage_to_letter_grade(numeric)	Immutable function: maps 0–100 percentage to letter grade string per Art. 17.

Function	Purpose
credits_to_level(int)	Immutable function: maps passed credit count to academic level enum per Art. 10.

7.4 Index Strategy

Index	Columns	Query Pattern Served
idx_enrollments_student_semester	(student_id, semester_id)	Schedule page, credit load calculation, GPA recalculation.
idx_enrollments_status	(status)	Dashboard active enrollment count; finalization batch.
idx_students_code	(student_code)	Admin search by student code.
idx_students_status	(academic_status)	Warning/dismissal reports and filters.
idx_notifications_unread	(user_id, is_read) WHERE is_read=FALSE	Partial index — unread badge count; very fast.
idx_offerings_semester	(semester_id)	Loading all offerings for a semester for registration page.
idx_courses_code	(code)	Prerequisite chain traversal by course code.
idx_warnings_student	(student_id)	Student warning history load on transcript page.
idx_gpa_student	(student_id)	GPA history trend chart on transcript page.

8. Deployment Architecture & Security

8.1 Service Dependency Chain

All five services are configured with Docker health checks and depends_on conditions enforcing strict startup ordering. No service starts until its dependency passes health checks.

Service	Pinned Image	Port	Depends On (condition)
postgres	postgres:16.3-alpine3.20	5432	— (first in chain)
backend	node:20-alpine (custom build)	3000	postgres: service_healthy
frontend	nginx-unprivileged:1.27-alpine	3001	backend: service_healthy
prometheus	prom/prometheus:v2.52.0	9090	backend: service_healthy
grafana	grafana/grafana:10.4.3	3050	prometheus: service_healthy

8.2 Security Controls Applied

Security Control	Implementation Detail
Password storage	bcrypt 10 rounds. 60-char hash. Never logged or transmitted in plain text.
Token expiry	Access: 15 min (limits blast radius). Refresh: 7 days stored as SHA-256 hash in DB.
Token rotation	On every /auth/refresh: old token revoked before new pair issued (prevents replay attacks).
Constant-time auth	Dummy bcrypt.compare() for non-existent users prevents timing-based user enumeration.
Rate limiting	Global: 200 req/15 min. Login: 10 req/15 min (express-rate-limit). Prevents brute force.
HTTP security headers	Helmet.js: Strict-Transport-Security, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, X-XSS-Protection, Content-Security-Policy.
CORS policy	Explicit origin whitelist via CORS_ORIGINS env var. Credentials allowed only for listed origins.
SQL injection prevention	100% parameterized queries ($1, $2 placeholders). No ORM, no string concatenation in SQL.
Non-root containers	Backend: nodejs:1001. Prometheus: nobody:65534. Grafana: grafana:472. Nginx: unprivileged image.

Security Control	Implementation Detail
Read-only mounts	database/ directory mounted :ro in backend container — seeds/schema cannot be modified at runtime.
Secrets management	JWT_SECRET and DB_PASSWORD via environment variables. Never hardcoded. Validated at startup.
Image pinning	All Docker images use exact version tags (no :latest) preventing unexpected upstream changes.

8.3 SDLC Methodology & Sprint Plan

The project followed an Agile-Waterfall hybrid. Requirements and architecture used a Waterfall approach (all 52 bylaw articles fully specified before coding). Implementation and testing used iterative 2-week sprints.

Phase	Duration	Key Deliverables
Requirements	Weeks 1–2	Bylaw analysis; FR/NFR specification; SRS feature matrix (60 features).
Architecture & DB Design	Weeks 3–4	Database schema (22 tables); DFD/UML models; API contract; bylaw constants.
Backend Implementation	Weeks 5–8	Node.js API (60 endpoints); PL/pgSQL functions; JWT auth; bylaw enforcement services.
Frontend Implementation	Weeks 7–10	React dashboards for all 3 roles; Axios interceptor; real-time GPA/checklist views.
Testing & QA	Week 11	Unit tests (GPA/bylaw services); integration tests (registration flow); load test 100 concurrent.
Deployment & DevOps	Week 12	Docker Compose; .dockerignore; Dockerfile optimization; Prometheus/Grafana; quick-start.sh.
Documentation	Week 13	This SDLC document; README; API reference; in-code comments.

9. References

#	Reference
1	Sommerville, I. (2016). Software Engineering (10th ed.). Pearson. — Chapters 4 (Requirements Engineering), 6 (Architectural Design), 7 (Design & Implementation).
2	Pressman, R. S., & Maxim, B. R. (2014). Software Engineering: A Practitioner's Approach (8th ed.). McGraw-Hill. — Chapters 5 (Agile), 7 (Requirements), 12 (Design Concepts).
3	Kendall, K. E., & Kendall, J. E. (2014). Systems Analysis and Design (9th ed.). Pearson. — Chapters 3 (Feasibility Study), 9 (Designing Databases), 10 (DFD Notation).
4	Fowler, M. (2003). UML Distilled: A Brief Guide to the Standard Object Modeling Language (3rd ed.). Addison-Wesley. — Class diagrams, sequence diagrams, use case notation.
5	Booch, G., Rumbaugh, J., & Jacobson, I. (2005). The Unified Modeling Language User Guide (2nd ed.). Addison-Wesley. — UML 2.x standard notation.
6	DeMarco, T. (1979). Structured Analysis and System Specification. Prentice-Hall. — DFD notation, levelling, and data dictionary conventions.
7	Elmasri, R., & Navathe, S. (2015). Fundamentals of Database Systems (7th ed.). Pearson. — ER modelling (Chen notation, Ch. 3–5), relational design theory.
8	Cockburn, A. (2000). Writing Effective Use Cases. Addison-Wesley. — Use case flow of events format, alternative flows.
9	IEEE Std 830-1998. IEEE Recommended Practice for Software Requirements Specifications. IEEE.
10	IEEE Std 1016-2009. IEEE Standard for Information Technology — Systems Design — Software Design Descriptions. IEEE.
11	Prime Minister Decree No. 2771 of 2018. Egypt — Governing Universities Regulation (Governing bylaws basis).
12	Ministerial Decree No. 4175 of 2019. Ministry of Higher Education, Egypt (Credit-hour system mandate).
13	Tanta University, Faculty of Computers and Informatics. (2024). Bachelor's Degree Program Bylaws — Academic Year 2024. Official Faculty Document.